

Convolutional Neural Networks

DL Specialization, Course 4 — detailed notes (Weeks 1–4)

Convolutions · classic & modern architectures · object detection · face recognition & style transfer

1. Week 1 — Foundations of ConvNets

1.1 Why not fully-connected

A $1000 \times 1000 \times 3$ image = 3M inputs. One FC hidden layer of 1000 units needs a $1000 \times 3\text{M} = 3$ billion-parameter matrix — infeasible. **Convolutions** share a small filter across the whole image, slashing parameters.

1.2 Edge detection = convolution

Slide a filter (kernel) over the image, multiply-and-sum at each position. A vertical-edge filter:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad (\text{Sobel: } 1, 2, 1 \text{ rows; Scharr: } 3, 10, 3).$$

Bright-to-dark region $\rightarrow$ positive response; dark-to-bright $\rightarrow$ negative (use $|\cdot|$). A 6×6 image $\ast 3 \times 3$ filter $\rightarrow 4 \times 4$. **Key DL idea:** don't hand-design the filter — *learn* its 9 numbers as weights, so the net discovers whatever edges/features help.

1.3 Padding

Convolution shrinks the output ($n \rightarrow n - f + 1$) and under-uses edge pixels. Pad with p zeros around the border:

$$\text{output} = (n + 2p - f + 1) \times (n + 2p - f + 1).$$

Valid conv: $p = 0$ (shrinks). **Same** conv: output size = input, needs $p = \frac{f-1}{2}$ (so f is usually odd).

1.4 Strided convolutions

Move the filter by stride s each step:

$$\text{output} = \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor.$$

(DL's "convolution" is technically cross-correlation — the filter isn't flipped; doesn't matter in practice.)

1.5 Convolutions over volume

An RGB image is $n \times n \times n_c$; the filter must match channels: $f \times f \times n_c$. Convolving gives a 2-D output ($4 \times 4 \times 1$). Use **multiple filters** to detect multiple features, stacking outputs along the channel axis:

$$6 \times 6 \times 3 \ast 10 \text{ filters of } 3 \times 3 \times 3 \Rightarrow 4 \times 4 \times 10 \quad (p=0, s=1).$$

1.6 One convolutional layer

Convolve, add a bias per filter, apply ReLU:

$$a^{[l]} = \text{ReLU}(W^{[l]} \ast a^{[l-1]} + b^{[l]}).$$

Parameter count is independent of input size — each filter is $f \times f \times n_c^{[l-1]}$, plus one bias. E.g. 10 filters of $3 \times 3 \times 3$: $(27 + 1) \times 10 = 280$ params, whatever the image size. This is why ConvNets resist overfitting.

Notation for layer l : filter size $f^{[l]}$, padding $p^{[l]}$, stride $s^{[l]}$, # filters $n_c^{[l]}$; output $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$.

1.7 Pooling layers

Downsample to shrink size, speed compute, and add translation robustness. **Max pooling**: take the max in each $f \times f$ window (typically $f = 2, s = 2 \Rightarrow$ halves H,W). **Average pooling**: mean instead. *No parameters to learn* (just hyperparameters f, s). Max pooling is far more common.

1.8 CNN example (LeNet-style) & why convolutions

Typical pattern: Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ Softmax. Input shrinks in H,W while channels grow; FC layers hold most of the parameters.

Two reasons convolutions work: (1) **parameter sharing** — a useful feature detector (e.g. an edge) is reused across the whole image; (2) **sparsity of connections** — each output depends on only a few inputs. Both cut parameters and give **translation invariance**.

2. Week 2 — Deep Convolutional Models (Case Studies)

2.1 Classic networks

- **LeNet-5** (1998): handwritten digits, $\sim 60k$ params. Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ out. H,W shrink, channels grow.
- **AlexNet** (2012): similar but $\sim 60M$ params, ReLU, trained on ImageNet — convinced the field that DL matters.
- **VGG-16** (2015): only 3×3 same-convs ($s=1$) and 2×2 max-pool ($s=2$); $\sim 138M$ params. Simple, uniform rule: channels double $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ as pooling halves H,W.

2.2 ResNets (residual networks)

Very deep plain nets are hard to train (vanishing/exploding gradients). A **residual block** adds a *skip connection* that bypasses two layers:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}).$$

Stacking these lets you train 100+ layers. In a plain net, training error *rises* again past some depth; with ResNets it keeps falling.

Why they work

If regularization drives $W^{[l+2]} \rightarrow 0$, the block computes $a^{[l+2]} = g(a^{[l]}) = a^{[l]}$ — the **identity** is easy to learn. So extra layers never *hurt*; they can only help. (If dimensions differ, use $a^{[l+2]} = g(z^{[l+2]} + W_s a^{[l]})$ with a learned/zero-padded W_s .) The skip also gives gradients a shortcut to backpropagate.

2.3 1×1 convolutions (Network in Network)

A $1 \times 1 \times n_c$ filter combines channels at each pixel — like a mini FC layer across channels.

Use to **shrink the channel count** (e.g. $28 \times 28 \times 192 \rightarrow 28 \times 28 \times 32$) cheaply, or add non-linearity while keeping H,W.

2.4 Inception network

“Why choose one filter size? Use them all.” An **inception module** runs 1×1 , 3×3 , 5×5 (same-conv) and max-pool in parallel, then concatenates channels — letting the net learn which to favour.

Cost problem & bottleneck fix: a direct 5×5 on $28 \times 28 \times 192 \rightarrow 32$ costs $\sim 120\text{M}$ mults. Insert a 1×1 “bottleneck” to 16 channels first: $2.5\text{M} + 10\text{M} \approx 12.5\text{M}$ — a $10\times$ saving, with no performance loss.

The full **GoogLeNet** stacks these modules, with side-branch softmaxes for regularization.

2.5 MobileNet (efficient, for mobile/embedded)

Replaces a normal conv with a **depthwise separable convolution** = two cheaper steps:

1. **Depthwise:** one $f \times f$ filter *per input channel* (no mixing across channels).
2. **Pointwise:** a $1 \times 1 \times n_c$ conv to mix channels and set the output depth.

Cost ratio vs. normal conv $\approx \frac{1}{n'_c} + \frac{1}{f^2}$ — roughly $10\times$ cheaper for typical sizes. **MobileNet v2** adds a residual (skip) connection and an “expansion” 1×1 before the depthwise step (*expand* $\rightarrow$ *depthwise* $\rightarrow$ *project/bottleneck*), raising accuracy while staying cheap.

2.6 EfficientNet

Systematically scales a network’s **resolution, depth, and width together** (compound scaling) to trade accuracy vs. compute — pick the point that fits your device’s budget.

2.7 Practical advice

- **Use open-source:** start from a published architecture + pretrained weights (GitHub) rather than from scratch.
- **Transfer learning:** download a net pretrained on ImageNet, replace the softmax, freeze early layers. Little data $\rightarrow$ freeze more (even precompute frozen activations to disk); more data $\rightarrow$ fine-tune more layers.
- **Data augmentation:** mirroring, random cropping, rotation/shearing, color shifting (PCA color augmentation). Run augmentation on CPU threads in parallel with training.
- **State of CV:** vision has relatively little data $\Rightarrow$ lots of hand-engineering and complex architectures. Benchmark tricks (not for production): ensembling, multi-crop at test time (e.g. 10-crop).

3. Week 3 — Object Detection

3.1 Localization & landmarks

Classification with localization: output class *and* a bounding box. Target vector:

$$y = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, \dots]^\top,$$

p_c = object present? If $p_c=0$, the rest is “don’t care.” **Landmark detection:** output fixed points (e.g. 64 face landmarks $l_{1x}, l_{1y}, \dots$) — basis for pose/AR filters. Loss: logistic for p_c , squared error

for boxes/landmarks.

3.2 Sliding windows — convolutional implementation

Naive sliding windows (crop, classify, repeat) is far too slow. **Turn FC layers into conv layers** (1×1 convs), then one forward pass over the whole image computes *all* window positions at once, sharing computation. Weakness: box positions are coarse.

3.3 YOLO — bounding box predictions

“You Only Look Once”: one CNN pass predicts boxes directly on a grid.

1. Overlay a grid (e.g. 19×19). Each cell outputs $[p_c, b_x, b_y, b_h, b_w, \text{classes}]$.
2. $b_x, b_y \in (0, 1)$ relative to the cell; b_h, b_w relative to the cell (can exceed 1).
3. Object assigned to the cell containing its *midpoint*. One pass $\Rightarrow$ fast, real-time.

3.4 Intersection over Union (IoU)

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}.$$

≥ 0.5 counts as a correct detection (1.0 = perfect). Used to evaluate and to filter.

3.5 Non-max suppression

Removes duplicate boxes for one object:

1. Discard boxes with $p_c < 0.6$.
2. Pick the highest- p_c box, output it.
3. Discard any remaining box with $\text{IoU} > 0.5$ vs. that one. Repeat.

Run once per class.

3.6 Anchor boxes

Let one grid cell detect *multiple* overlapping objects (e.g. person + car). Define k anchor shapes; the target stacks one box-set per anchor. Each object goes to (cell, anchor) with highest IoU. Anchors let the net specialize (tall vs. wide objects); shapes chosen by hand or by k-means.

3.7 YOLO algorithm (full) & region proposals

Label y shape = $[\text{grid}_H, \text{grid}_W, \# \text{anchors} \times (5 + \# \text{classes})]$. Train, then at test run non-max suppression per class. **R-CNN family** (optional): propose regions (segmentation) then classify — *Fast / Faster* R-CNN speed it up, but YOLO (one pass) is generally faster.

3.8 Semantic segmentation with U-Net

Label *every pixel* with a class (not just boxes). Needs to output a full-resolution map, so the net must *up-sample*.

- **Transpose convolution:** the “reverse” of conv — takes a small input, spreads each value through a filter onto a larger output (uses f, p, s to grow H, W).
- **U-Net:** an encoder (down-sampling convs) then a decoder (transpose convs), with **skip connections** copying high-res features from encoder to decoder — so fine spatial detail (from early layers) combines with semantic context (from deep layers). Output: $H \times W \times \# \text{classes}$.

4. Week 4 — Face Recognition & Neural Style Transfer

4.1 Face recognition — verification vs. recognition

Verification (1:1): is this the claimed person? **Recognition** (1:K): who, among K people, is this (or none)? Recognition is harder — needs very high verification accuracy since errors compound over K.

4.2 One-shot learning & similarity function

You often have *one* image per person — too few to train a classifier. Instead learn a **similarity function**:

$$d(\text{img}_1, \text{img}_2) = \text{degree of difference}; \quad d \leq \tau \Rightarrow \text{same person.}$$

Robust to new people without retraining.

4.3 Siamese network

Encode each image into a vector $f(x)$ with the *same* conv net (shared weights); compare encodings:

$$d(x_1, x_2) = \|f(x_1) - f(x_2)\|_2^2.$$

Train so same-person pairs are close, different-person pairs far.

4.4 Triplet loss

Train on triplets: **A**nchor, **P**ositive (same person), **N**egative (different):

$$\begin{aligned} \|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha &\leq 0, \\ L(A, P, N) &= \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0). \end{aligned}$$

α = margin (stops the trivial all-zeros solution). Choose *hard* triplets (where $d(A, P) \approx d(A, N)$) so training isn't too easy. (FaceNet.)

4.5 Face verification as binary classification

Alternative to triplet loss: feed two encodings into a logistic unit:

$$\hat{y} = \sigma\left(\sum_k w_k |f(x^{(i)})_k - f(x^{(j)})_k| + b\right).$$

Same Siamese (shared) encoder. **Deployment trick**: precompute encodings for enrolled people; at test only encode the new image and compare.

4.6 Neural style transfer — what ConvNets learn

Generate image G with the *content* of C and the *style* of S . First, visualize what deep ConvNets learn: shallow layers detect edges/colors; deeper layers detect complex patterns/objects (find the image patches that maximize each unit).

4.7 Cost function

$$J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G).$$

Initialize G randomly, then *gradient-descent on the pixels of G* (the network's weights are fixed):
 $G := G - \frac{\partial J}{\partial G}.$

Content cost

Use a hidden layer l (mid-depth) of a pretrained net (e.g. VGG). If $a^{[l](C)}$ and $a^{[l](G)}$ are similar, contents match:

$$J_{\text{content}} = \frac{1}{2} \|a^{[l](C)} - a^{[l](G)}\|^2.$$

Style cost — Gram matrix

Define “style” as *correlations between channels* of a layer’s activations. The **Gram (style) matrix** $G^{[l]} \in \mathbb{R}^{n_c \times n_c}$:

$$G_{kk'}^{[l]} = \sum_i \sum_j a_{ijk}^{[l]} a_{ijk'}^{[l]} \quad (\text{compute for } S \text{ and } G),$$
$$J_{\text{style}}^{[l]} = \frac{1}{(2n_H n_W n_c)^2} \|G^{[l](S)} - G^{[l](G)}\|_F^2, \quad J_{\text{style}} = \sum_l \lambda^{[l]} J_{\text{style}}^{[l]}.$$

Summing style cost over several layers gives richer style. (Efficiently: reshape activations to $HW \times n_c$, then $G = F^\top F$.)

4.8 1D and 3D generalizations

Convolution isn’t just 2-D. **1-D**: a length- n signal $\ast$ length- f filter (EKG, audio) — output $\frac{n-f}{s} + 1$ (though sequences often use RNNs). **3-D**: volumes like CT scans ($14 \times 14 \times 14 \times 1$ * filters) — same $(n-f)/s + 1$ rule per dimension.

Big picture of Course 4: Week 1 builds the conv/pool primitives (parameter sharing = few params, translation invariance). Week 2 shows how to *compose* them — skip connections (ResNet) to go deep, 1×1 bottlenecks (Inception) and depthwise-separable convs (MobileNet) to stay cheap — and to reuse others’ nets via transfer learning. Week 3 turns classification into *detection* (YOLO: grid + anchors + IoU + non-max suppression) and *segmentation* (U-Net: encode–decode with skips). Week 4 reuses a conv net as a learned *encoder* — for face similarity (Siamese + triplet loss) and for style transfer (optimize the image, not the weights).